

Beacon Developer API Quickstart and Webhook Verification

Doc ID	DEV-API-2026-06
Version	v4.1
Owner	Developer Platform
Document owner	Theo Brandt, Platform Engineering
Last reviewed	2026-06-30
Classification	Customer
Related	Data Retention by Plan (CS-RET-2026-01), Campaign Analytics Glossary (CS-ANL-2025-10)

What the API is for

The Beacon Developer API is a REST API for syncing contacts, tracking events, triggering messages, and reading analytics from your own systems. Webhooks push delivery and engagement events to an endpoint you control as they happen. Together they let you treat Beacon as a messaging layer inside your product rather than a tool your marketing team logs into.

The Developer API is an Enterprise product at \$299 per month with no seat limit. It sends on email, SMS, and push. In-app messages are delivered through the Channels product and its client SDK, not through the REST API.

1. Create an API key

API keys are created by workspace Admins in Administration → API keys. Editors and Viewers cannot create or view keys.

1. Choose **Create key**.
2. Name it for the system that will use it (`checkout-service` , not `Theo's key`).
3. Choose scopes. Start with the minimum: `contacts:write` and `events:write` for a typical product integration.
4. Copy the key. Beacon shows it once. If you lose it, revoke and create a new one.

Keys are prefixed by environment: `bk_live_` for production and `bk_test_` for the sandbox. Sandbox keys accept every call, deliver nothing, and do not meter usage.

Store keys in your secret manager. Never ship a key in client-side code, a mobile app bundle, or a public repository. Beacon scans public repositories for leaked keys and revokes any it finds, then emails your Admins.

Scopes

Scope	Allows
contacts:read	Read contacts and segment membership
contacts:write	Create, update, delete contacts
events:write	Track custom events
messages:send	Trigger transactional messages
analytics:read	Read campaign and workflow metrics
webhooks:manage	Create and modify webhook endpoints

2. Make your first call

All requests use HTTPS to `https://api.beacon.example/v1` with a Bearer token.

```
curl https://api.beacon.example/v1/contacts \
-H "Authorization: Bearer bk_test_..." \
-H "Content-Type: application/json" \
-d '{
  "external_id": "user_8213",
  "email": "ana@example.com",
  "attributes": {"plan": "growth", "signup_source": "web"},
  "consent": {"email": "opted_in", "sms": "unknown"}
}'
```

A `201 Created` returns the contact with its Beacon ID. Sending the same `external_id` again updates the contact instead of duplicating it. Use your own user ID as `external_id` from day one; it makes every later sync idempotent.

Always send consent. A contact created without an email consent status cannot be targeted by email campaigns, and your marketing team will open a ticket asking why the segment is empty.

3. Track an event

```
curl https://api.beacon.example/v1/events \
-H "Authorization: Bearer bk_test_..." \
-H "Content-Type: application/json" \
-d '{
  "external_id": "user_8213",
  "event": "order_completed",
  "timestamp": "2026-06-30T14:02:11Z",
  "properties": {"order_id": "A-10422", "value": 84.50, "currency": "USD"}
}'
```

Events drive three things: they can trigger a workflow, they can be used as segment filters ("did `order_completed` in the last 30 days"), and they can serve as conversion goals in Analytics. A `value` property is summed as revenue when the event is a goal.

Event names are case-sensitive and become permanent once used. Agree on a naming convention before the first call. Beacon recommends `object_verb` in lower snake case.

Batch up to 1,000 events per request at `/v1/events/batch`. Batching counts as one request against your rate limit.

4. Rate limits

The default limit is **100 requests per second** per project. When you exceed it, Beacon returns `429 Too Many Requests` with a `Retry-After` header giving the number of seconds to wait. Respect the header. Clients that retry immediately are throttled further.

Every response includes `X-RateLimit-Limit`, `X-RateLimit-Remaining`, and `X-RateLimit-Reset`. Watch `Remaining` and slow down before you hit zero rather than after.

If your integration needs more than 100 rps sustained, use the batch endpoints first. Most integrations that hit the limit are sending one event per request when they could send a thousand. If batching is not enough, contact Support with your projected volume; higher limits are available on Enterprise agreements.

5. Trigger a transactional message

Transactional messages (receipts, password resets, alerts) are sent from templates you create in Campaigns → Templates and trigger from the API with `messages:send`.

```
curl https://api.beacon.example/v1/messages \
-H "Authorization: Bearer bk_live_..." \
-H "Content-Type: application/json" \
-d '{
  "template": "order_receipt",
  "external_id": "user_8213",
  "channel": "email",
  "data": {"order_id": "A-10422", "items": 3}
}'
```

Transactional messages ignore marketing consent but do respect hard bounces and complaints. They are metered as message volume like any other delivery.

6. Webhooks

Webhooks deliver events from Beacon to you: `message.sent`, `message.delivered`, `message.bounced`, `message.opened`, `message.clicked`, `contact.unsubscribed`, `contact.complained`, `workflow.completed`, and others. Create an endpoint in Administration → Webhooks or through the API with `webhooks:manage`.

Each webhook request is a `POST` with a JSON body and these headers:

Header	Purpose
Beacon-Signature	HMAC-SHA256 of the timestamp and body, hex encoded
Beacon-Timestamp	Unix seconds when Beacon signed the request
Beacon-Event-Id	Unique ID for deduplication
Beacon-Event-Type	The event name

Verifying the signature

Every endpoint has a signing secret, shown once when you create it and rotatable at any time. Verify every request before you trust it.

1. Read the raw request body as bytes. Do not parse and re-serialize it; whitespace changes break the signature.
2. Build the signed payload: `{Beacon-Timestamp}.{raw body}`.
3. Compute HMAC-SHA256 of the signed payload using your signing secret as the key.
4. Compare the hex digest to `Beacon-Signature` using a constant-time comparison.
5. Reject requests whose timestamp is more than five minutes old. This limits replay.

```
import hmac, hashlib, time

def verify(secret: str, timestamp: str, body: bytes, signature: str) -> bool:
    if abs(time.time() - int(timestamp)) > 300:
        return False
    payload = timestamp.encode() + b"." + body
    expected = hmac.new(secret.encode(), payload, hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, signature)
```

During a secret rotation, Beacon signs with both the old and new secret for 24 hours and sends both signatures separated by a comma in `Beacon-Signature`. Accept the request if either matches.

Delivery and retries

Respond with any `2xx` within 10 seconds. Anything else, or a timeout, is treated as a failure and Beacon retries with exponential backoff for up to 24 hours: roughly 1 minute, 5 minutes, 30 minutes, 2 hours, 6 hours, then every 6 hours.

Retries mean you will sometimes receive the same event twice. Use `Beacon-Event-Id` to deduplicate. Process the event after you respond, not before; queue it and return `200` immediately.

After 24 hours of continuous failure, Beacon disables the endpoint and emails your Admins. Re-enable it in [Administration](#) → [Webhooks](#). Events missed while disabled can be replayed for the last 7 days from the same page.

Ordering

Webhooks are not guaranteed to arrive in order. A `message.opened` can arrive before its `message.delivered`. Use the `occurred_at` field in the body, not arrival time, when ordering matters.

7. Reading analytics

With `analytics:read`, pull campaign and workflow metrics from `/v1/analytics/campaigns/{id}` and `/v1/analytics/workflows/{id}`. Metrics follow the definitions in the Campaign Analytics Glossary (CS-ANL-2025-10). Contact-level event history at `/v1/contacts/{id}/events` is limited by your plan's retention period; see Data Retention by Plan (CS-RET-2026-01). To keep more history than your plan retains, consume webhooks into your own warehouse.

8. Errors

Status	Meaning	What to do
400	Malformed request	Read the <code>errors</code> array; each entry names a field
401	Missing or invalid key	Check the header and the key's environment prefix
403	Key lacks the scope	Add the scope or use a different key
404	Unknown contact, template, or endpoint	Check IDs; contacts are looked up by <code>external_id</code>
409	Conflict, usually a duplicate <code>external_id</code> on create	Use update semantics
422	Valid JSON, invalid values	Consent values, channel names, and event names are enumerated
429	Rate limited	Wait for <code>Retry-After</code>
5xx	Beacon-side error	Retry with backoff; check the status page

Every error body includes a `request_id`. Include it when you contact Support.

9. Checklist before going live

Done	Item
	Production key stored in a secret manager, scoped to what the service needs
	<code>external_id</code> set to your own user ID on every contact
	Consent sent on every contact create
	Event naming convention agreed and documented
	Batch endpoint used for backfills
	429 handling honors <code>Retry-After</code>
	Webhook signature verified with constant-time compare and timestamp check
	Webhook handler returns 200 before processing and deduplicates on <code>Beacon-Event-Id</code>

Revision history

Version	Date	Change
v3.0	2025-03-19	Added batch endpoints and scopes table
v4.0	2025-12-10	Rewrote webhook section; added rotation and replay
v4.1	2026-06-30	Added error table and go-live checklist